

Relatório Técnico: Resolução do Problema da Mochila 0-1 usando CPLEX Concert C++

Caio Anderson Bezerra Fernandes

13 de julho de 2026

1 Introdução

O Problema da Mochila 0-1 (0-1 Knapsack Problem) é um clássico problema de otimização combinatória NP-difícil. Dado um conjunto de itens, cada um com um peso e um valor (lucro), o objetivo é selecionar um subconjunto de itens de forma a maximizar o valor total acumulado sem que a soma dos pesos dos itens selecionados ultrapasse a capacidade máxima da mochila.

2 Formulação Matemática

Seja N o número total de itens disponíveis e W a capacidade máxima da mochila. Para cada item $i \in \{1, \dots, N\}$, define-se:

- v_i : o valor (lucro) do item i ($v_i \geq 0$).
- w_i : o peso do item i ($w_i \geq 0$).

O problema pode ser formulado como um modelo de Programação Linear Inteira Binária (PLIB) através da definição das variáveis de decisão:

$$x_i = \begin{cases} 1, & \text{se o item } i \text{ for selecionado para a mochila} \\ 0, & \text{caso contrário} \end{cases}$$

A formulação matemática é expressa por:

$$\text{Maximizar } \sum_{i=1}^N v_i x_i \tag{1}$$

$$\text{Sujeito a } \sum_{i=1}^N w_i x_i \leq W \tag{2}$$

$$x_i \in \{0, 1\}, \quad \forall i \in \{1, \dots, N\} \tag{3}$$

Onde:

- A Equação (1) define a função objetivo, maximizando o lucro acumulado dos itens selecionados.
- A Restrição (2) garante que a soma dos pesos dos itens inseridos na mochila não exceda a capacidade limite W .
- A Restrição (3) impõe que a variável de escolha seja binária (decisão 0-1).

3 Implementação Computacional em C++

A seguir é apresentado o trecho principal do solver implementado no arquivo `mochila_solver.cpp`, utilizando a API CPLEX Concert em C++:

```
1 class KnapsackSolver {
2 public:
3     static void solve(const KnapsackInstance& instance) {
4         int N = instance.get_N();
5         double W = instance.get_capacity();
6         IloEnv env;
7
8         try {
9             IloModel model(env);
10
11             // Variaveis de decisao: x[i] = 1 se selecionado, 0 caso
contrario
12             IloBoolVarArray x(env, N);
13
14             // Funcao Objetivo: Maximizar o lucro total
15             IloExpr obj_expr(env);
16             for (int i = 0; i < N; ++i) {
17                 obj_expr += instance.get_value(i) * x[i];
18             }
19             model.add(IloMaximize(env, obj_expr));
20             obj_expr.end();
21
22             // Restricao de Capacidade da Mochila: peso total <= W
23             IloExpr weight_expr(env);
24             for (int i = 0; i < N; ++i) {
25                 weight_expr += instance.get_weight(i) * x[i];
26             }
27             model.add(weight_expr <= W);
28             weight_expr.end();
29
30             IloCplex cplex(model);
31             cplex.setOut(env.getNullStream()); // Oculta o log padrao do
CPLEX
32             cplex.setParam(IloCplex::Param::Threads, 1);
33
34             // Exportacao do modelo matematico para depuracao em formato
LP
35             cplex.exportModel("knapsack_model.lp");
36
37             auto start_time = std::chrono::high_resolution_clock::now();
38             if (!cplex.solve()) {
39                 std::cerr << "Modelo inviavel ou falha no solver." <<
std::endl;
40                 env.end();
41                 return;
42             }
43             auto end_time = std::chrono::high_resolution_clock::now();
44             double elapsed = std::chrono::duration<double>(end_time -
start_time).count();
45
46             // Recuperacao e exibicao de estatisticas finais
```

```

47     double total_value = cplex.getObjValue();
48     double total_weight = 0.0;
49     std::vector<int> selected_items;
50     for (int i = 0; i < N; ++i) {
51         if (cplex.getValue(x[i]) > 0.5) {
52             selected_items.push_back(i);
53             total_weight += instance.get_weight(i);
54         }
55     }
56     // Exibicao dos resultados omitida para concisao...
57 } catch (IloException& e) {
58     std::cerr << "Erro no Concert: " << e << std::endl;
59 }
60 env.end();
61 }
62 };

```

Listing 1: Classe KnapsackSolver com formulação CPLEX Concert

4 Formatos de Instâncias Suportados

O resolvidor foi dotado de um parser robusto que realiza a detecção explícita e automática de dois tipos comuns de instâncias:

4.1 Formato de Cabeçalho Estruturado (Chave-Valor)

Este formato é caracterizado por metadados descritos explicitamente e um marcador obrigatório delimitando o início da listagem de itens (ITEMS ou DATA). Permite ainda nomes de itens contendo espaços:

```
N: 4
CAPACITY: 100
ITEMS
Item Super Premium A 50 40
Item_B 30 20
```

4.2 Formato Padrão da Literatura (KPLIB)

Compatível com as instâncias oficiais da KPLIB (Knapsack Problem Library), sem strings de marcação ou delimitadores chave-valor. O arquivo é composto unicamente por inteiros e reais:

```
50
14778
845 804
758 448
```

5 Resultados Computacionais

O solver foi submetido a baterias de testes com instâncias reais contendo 1.000 itens retiradas da biblioteca benchmark KPLIB. Os tempos de execução refletem o poder de otimização combinatória do resolvidor com CPLEX em hardware moderno:

6 Repositórios e Recursos

Os códigos-fonte desenvolvidos e as bases de instâncias utilizadas neste trabalho estão disponíveis nos seguintes endereços eletrônicos:

- **Código-Fonte em C++ (CPLEX):** Repositório com a implementação completa do resolvidor, instruções de compilação e exemplos de execução.
<https://github.com/caioandersonf/cplex-aplicado-ao-problema-da-mochila>
- **Biblioteca de Instâncias (KPLIB):** Repositório contendo o conjunto completo de instâncias de benchmark da literatura do problema da mochila 0-1.
<https://github.com/likr/kplib>

Tabela 1: Resultados dos testes computacionais na base KPLIB ($N = 1000$)

Categoria da Instância	Lucro Ótimo	Capacidade Utilizada / Limite	Tempo (s)
Uncorrelated	400.792	252.461 / 252.480	0,0096 s
Weakly Correlated	273.039	245.968 / 245.972	0,0140 s
Strongly Correlated	316.372	245.972 / 245.972	0,0035 s
Inverse Strongly Correlated	263.977	295.477 / 295.477	0,0032 s
Almost Strongly Correlated	316.408	245.972 / 245.972	0,0049 s
Subset Sum	245.972	245.972 / 245.972	0,0025 s
Uncorrelated Similar Weights	371.246	49.525.300 / 49.530.200	0,0025 s
Spanner Uncorrelated	228.624	84.656 / 84.660	0,0009 s
Spanner Weakly Correlated	196.050	198.664 / 198.695	0,0015 s
Spanner Strongly Correlated	488.704	198.804 / 198.807	0,0009 s
Multiple Strongly Correlated	399.170	245.970 / 245.970	0,0045 s
Profit Ceiling	245.874	245.971 / 245.972	0,0025 s
Circle	5.182.850	245.972 / 245.972	0,0025 s